

Benchmarking LLM-Augmented Web Vulnerability Detection

Juho Choi
University of Waterloo
j257choi@uwaterloo.ca

Raghav Mehta
University of Waterloo
r5mehta@uwaterloo.ca

Advait Sangle
University of Waterloo
asangle@uwaterloo.ca

Louis Song
University of Waterloo
m62song@uwaterloo.ca

Tengyi Xu
University of Waterloo
t48xu@uwaterloo.ca

1 Introduction and Motivation

Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) are the established automated methods for detecting vulnerabilities in web applications, operating on source code and on runtime behavior respectively. Both produce high false-positive rates, so their findings require manual triage before they are actionable.

Large language models (LLMs) have recently been applied to the same task, and they shift this trade-off rather than removing it. Recent comparisons report that LLMs detect a far higher fraction of vulnerabilities than SAST tools, in some settings 90–100%, but at a much higher false-positive rate, and that combining the two recovers some of each approach’s weaknesses [1]. This leaves a practical security engineering problem for anyone assembling a detection pipeline today: given SAST, DAST, plain LLMs, scanner-assisted LLMs, and agentic configurations, which approaches and combinations are worth using, and at what cost in false positives, compute, and latency?

This project investigates that problem empirically. We assemble the main detection approaches together with a range of LLM strategies (prompting variants, scanner-assisted triage, and agentic configurations that operate the scanners) and evaluate them against each other on the same vulnerable applications, under one scoring procedure with known ground truth, to give a clear, realistic account of the accuracy, false-positive, and cost trade-offs across these options.

2 Related Work

Prior work shows concretely how pairing an LLM with a scanner can improve on either tool alone. LLM4SA feeds SAST warnings and surrounding code to an LLM to filter false positives, reporting 81% precision on the Juliet benchmark and real C/C++ projects [2]; IRIS instead combines an LLM with CodeQL for whole-repository taint analysis and, on CWE-Bench-Java, detects 55 vulnerabilities against CodeQL’s 27 while lowering the false discovery rate [3]; and ZeroFalse extends warning filtering with flow-sensitive traces and CWE-specific context [4]. Each configuration pairs an LLM with a static tool, and each improves on the tool alone, so a static baseline in our evaluation should include not just a scanner and an unaided model but this kind of pairing. How close a pairing can get to outright replacement is shown by a recent study that compares thirty LLMs against Semgrep on OWASP Benchmark v1.2, where Semgrep scores F1 0.66 and the best model reaches 0.88 [5]; this is the closest existing

setup to our static baseline, though it evaluates LLMs as a replacement for the scanner rather than in combination with it.

Two further results narrow the design on the orchestration and dynamic sides. Sifting the Noise evaluates three agent frameworks—Aider, OpenHands, and SWE-agent—for SAST false-positive filtering and finds that agents can reduce the initial 92% false-positive rate on the OWASP Benchmark to as low as 6%, but that the gains over plain prompting are strongly backbone-dependent: stronger models benefit substantially from agentic scaffolding while weaker ones show limited or inconsistent improvement [6]. On the dynamic side, a constrained LLM layered atop conventional web scanners as a post-scan reasoner—with no direct tool calls—raised micro-averaged F1 from 0.58 to 0.82 and cut the false-alarm rate from 23.4% to 7.2% on WebGoat, DVWA, and bWAPP; its authors likewise found that a tightly scoped assistant outperformed an autonomous agent [7]. Both results point in the same direction: the gain from added orchestration complexity is real but conditional, not guaranteed. Our evaluation therefore places scanner-assisted pairings at the core, treats agentic and multi-agent configurations as conditions to be tested against that core rather than assumed winners, and compares static and dynamic approaches side by side—something prior work has not done.

3 Proposed Work

We evaluate the main detection approaches and a set of LLM strategies against each other under one scoring procedure. The starting configurations are:

- **A1 — Semgrep only** (SAST baseline).
- **A2 — OWASP ZAP only** (DAST baseline).
- **A3 — LLM only** (unaided model given the code or running app).
- **B1 — LLM + Semgrep output** (model receives SAST findings).
- **B2 — LLM + ZAP output** (model receives DAST findings).
- **B3 — LLM-authored Semgrep rules** (model writes or refines rules; Semgrep then runs them deterministically).
- **C1 — Multi-agent configurations** (specialized agent roles, such as a scanning agent paired with a separate verifier agent).

We treat these as starting points and experiment with the LLM strategies that matter most in practice: prompt

design, how scanner output is presented to the model, how the model can improve the scanner itself by authoring rules, and, as an exploratory extension, composing specialized agent roles (for example a scanning agent and a separate verifier agent) around the scanners. The evaluation runs through a single harness with a pluggable model backend, so each configuration can be driven by a local model or, with an API key, a hosted one, which also lets us compare a local model against a frontier model on identical inputs. Configurations are scored on the OWASP Benchmark, whose labeled test cases give automated ground truth and which is scored identically for SAST and DAST. A set of realistic vulnerable applications (OWASP Juice Shop, WebGoat, DVWA) serves as a secondary, qualitative target to see how the approaches transfer beyond the synthetic benchmark. Each configuration is measured on detection accuracy (precision, recall, F1), false-positive rate, and per-run cost and latency, so the comparison reflects engineering trade-offs rather than detection in isolation. The research questions are framed around those trade-offs:

- **RQ1.** Run alone (A1, A2, A3), how do SAST, DAST, and an unaided LLM compare on detection accuracy against false-positive rate and cost?
- **RQ2.** Does pairing an LLM with a scanner, whether by triaging its output (B1, B2) or by authoring its rules (B3), improve the accuracy versus false-positive trade-off over the scanner alone, and is the added cost justified?
- **RQ3.** What do more advanced AI strategies—including multi-agent orchestration, chain-of-thought prompting, and retrieval-augmented configurations—add to detection accuracy and false-positive control beyond the scanner-assisted pairings in RQ2, and under what conditions does the added complexity justify the cost?

To bound scope, the multi-agent configuration uses a fixed, small set of agent roles, and all scored runs use one designated machine and one model so that configurations remain directly comparable.

4 Tasks and Timeline

The project runs from June 19 to the August 14 final report, with the midterm report due July 17 and the project presentation on July 30.

- **Week 1 (Jun 19–26):** Finalize scope, metrics, and ground truth. Stand up the environment (Docker, OWASP Benchmark, Juice Shop, Semgrep, ZAP) and fix the configuration set.
- **Week 2 (Jun 27–Jul 3):** Run and validate the baselines (A1, A2, A3); build the scoring harness against ground truth.
- **Week 3 (Jul 4–10):** Build the LLM harness and the scanner-assisted configurations (B1, B2).
- **Week 4 (Jul 11–17):** Add LLM rule authoring (C3); first combined results. *Midterm report due July 17.*

- **Week 5 (Jul 18–24):** Full configuration runs on OWASP Benchmark (Java), plus the held-out Python Benchmark.
- **Week 6 (Jul 25–30):** Analysis and figures. *Project presentation July 30.*
- **Weeks 7–8 (Jul 31–Aug 14):** Qualitative runs on the realistic apps and the agentic demo; final analysis and writeup. *Final report due August 14.*

5 Work Delegation

- **Juho Choi:** Baseline scanners. Configures and runs Semgrep (SAST) and OWASP ZAP (DAST), producing baselines A1 and A2.
- **Raghav Mehta:** Evaluation methodology and scoring. Defines the configuration set and metrics, implements the ground-truth scoring scripts, and runs the comparative analysis.
- **Advait Sangle:** LLM and agentic harness. Builds the harness that supplies scanner output to the model, implements B1, B2, B3, and the multi-agent configuration C1, and the orchestration logic.
- **Louis Song:** LLM-only baseline and reporting. Implements the unaided-model configuration A3, and assembles the figures, tables, written report, and presentation.
- **Tengyi Xu:** Datasets and infrastructure. Containerized environment, target applications, reproducible run configuration, and ground-truth label management.

References

- [1] “Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection,” 2024. arXiv:2407.16235.
- [2] C. Wen, Y. Cai, B. Zhang, J. Su, Z. Xu, D. Liu, S. Qin, Z. Ming, and T. Cong, “Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?,” *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 7, 2024.
- [3] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities,” in *ICLR*, 2025. arXiv:2405.17238.
- [4] “ZeroFalse: Improving Precision in Static Analysis with LLMs,” 2025. arXiv:2510.02534.
- [5] D. Koterba, M. Łukaczyk, and W. Książek, “Leveraging Large Language Models for Advanced Static Code Analysis: Assessing the Feasibility of AI Superseding Traditional Vulnerability Detection Methods,” *Information and Software Technology*, 2026. doi:10.1016/j.infsof.2026.108213.
- [6] “Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering,” 2026. arXiv:2601.22952.
- [7] V. S. Gaikwad, M. Kulkarni, S. Deshmukh, J. Mehta, and P. Akolkar, “Penetration Testing for Websites using LLM,” Research Square preprint, 2026. doi:10.21203/rs.3.rs-9937475/v1.
- [8] OWASP Foundation, “OWASP Benchmark Project.”
- [9] OWASP Foundation, “OWASP Juice Shop.”